Placement de VNF & Optimisation des chaînes de services

De l'ordonnancement classique à la virtualisation réseau éco-énergétique

Année académique 2025–2026 | Équipes de 2 à 3 étudiants

L'objectif de ce projet est d'appliquer des méthodes de **recherche opérationnelle** à un problème réel issu des réseaux informatiques : le déploiement de **fonctions réseau virtualisées (VNF)** sur des serveurs physiques, tout en optimisant les performances des chaînes de services et l'efficacité énergétique.

Pour chaque phase, vous devrez :

- Comprendre le problème d'optimisation et son contexte réseau.
- Formuler le problème (définition des données, variables de décision, contraintes et fonction objectif).
- Implémenter le modèle ou l'algorithme en **Python** avec `gurobipy`.
- Reporter la valeur de la fonction objectif ainsi que les solutions optimales (ou les meilleures solutions obtenues).

Remarque :

- La documentation Gurobi et des exemples Python sont disponibles sur : gurobi.com/documentation.
- Une licence académique gratuite est disponible sur : gurobi.com/academia.
- Les instances de benchmark (SALBPGen) sont disponibles sur : assembly-line-balancing.de ou via le script générateur fourni sur la plateforme du cours.

Rendu : Soumettre une archive ZIP unique contenant :

(1) L'ensemble du code source, avec un fichier `README` expliquant comment l'exécuter.

(2) Un rapport PDF (un par équipe).

(3) Les tableaux de résultats au format `.csv`.

(4) Le projet Overleaf (les fichiers sources)

Utiliser **Python 3.10+** avec `gurobipy` et `numpy`. Aucun autre solveur n'est autorisé.

Évaluation :

Composante	Points	Critères principaux
Phase 1 — Implémentation MILP & benchmarking	5	Exactitude, tableau de résultats, analyse du solveur
Phase 2A — Trois heuristiques / métaheuristiques	5	Qualité, diversité, évaluation, réglage des paramètres
Phase 3 — Formulation & résolution EVNF-P	5	Modèle, validation, benchmarking, analyse des compromis
Qualité du rapport	3	Clarté, structure, figures, analyse critique
Qualité du code	2	Lisibilité, documentation, reproductibilité
Total	**20**	

Vous pouvez télécharger le template Overleaf à partir de ce lien.

Phase 1 — Implémentation et évaluation du modèle MILP de placement de VNF

1 Contexte : NFV, SFC et problème de placement

1.1 Virtualisation des fonctions réseau

Traditionnellement, les opérateurs réseau déployaient chaque fonction réseau — pare-feu, NAT, inspection approfondie des paquets, répartiteur de charge — sur des équipements matériels dédiés.

La NFV découple les fonctions réseau du matériel propriétaire en les exécutant sous forme de **fonctions réseau virtualisées (VNFs)** sur des serveurs standards, dans un centre de données ou en périphérie du réseau.

Les avantages sont significatifs : déploiement plus rapide, élasticité, réduction des coûts d'investissement, et capacité à instancier de nouvelles chaînes de services à la demande. La NFV constitue un pilier des réseaux cœur 5G, des architectures SD-WAN et des infrastructures réseau cloud-native. Le marché mondial de la NFV était évalué à plus de $25 milliards en 2023 et devrait dépasser $80 milliards d'ici 2030.

1.2 Chaînage de fonctions de service

Une **chaîne de fonctions de service (Service Function Chain, SFC)** spécifie une liste ordonnée de VNFs que doit traverser un flux de trafic. Par exemple, un trafic d'entreprise peut suivre la séquence suivante :

L'ordre de la chaîne n'est pas toujours strictement linéaire. En général, des **dépendances d'ordre partiel** apparaissent : l'IDS doit analyser un paquet déchiffré, il doit donc être placé après le terminateur TLS ; le répartiteur de charge doit intervenir après que le pare-feu a validé le flux — mais deux VNFs de monitoring indépendantes peuvent ne présenter aucune contrainte d'ordre entre elles. Cet ordre partiel est naturellement modélisé par un graphe orienté acyclique (DAG).

1.3 Le problème de placement des VNF

Étant donné m serveurs physiques et un ensemble de n modules VNF à déployer, l'opérateur doit affecter chaque VNF à exactement un serveur. Chaque VNF j requiert t_j unités de capacité de traitement (cycles CPU par paquet, ou équivalent en latence de traitement en microsecondes).

La charge d'un serveur est la somme des charges de traitement qui lui sont affectées. La latence goulot de la chaîne — c'est-à-dire le temps de traitement du serveur le plus chargé — détermine le débit de bout en bout de la chaîne en régime permanent.

L'opérateur souhaite **minimiser cette latence goulot** tout en respectant l'ordre imposé par le DAG et une infrastructure fixe de m serveurs. Il s'agit précisément du problème classique *Simple Assembly Line Balancing Problem de type 2* (SALBP-2), reformulé dans un contexte de virtualisation réseau.

Correspondance des concepts : production ↔ NFV

Concept SALBP-2	Concept NFV/SFC	Notation
Ligne d'assemblage	Chaîne de fonctions (SFC)	—
Poste k	Serveur physique / nœud de calcul	$k \in \mathcal{K}$
Tâche j	Module VNF (FW, NAT, IDS...)	$j \in \mathcal{J}$
Temps opératoire t_j	Charge CPU de la VNF j (μs/pkt)	$t_j > 0$
Arc de précédence (i, j)	Dépendance dans le plan de données (DAG)	$(i, j) \in P$
Charge du poste L_k	Utilisation du serveur	L_k
Temps de cycle CT	Latence goulot de la chaîne	CT
Minimiser CT	Minimiser le goulot, maximiser le débit	objectif

2 Phase 1 — Implémentation et évaluation du modèle MILP de placement de VNF

2.1 Variables de décision et notations

Ensembles et indices	
$\mathcal{J} = \{1, \ldots, n\}$	Ensemble des modules VNF
$\mathcal{K} = \{1, \ldots, m\}$	Ensemble des serveurs physiques
$P \subseteq \mathcal{J} \times \mathcal{J}$	Arcs de dépendance : $(i, j) \in P$ signifie que la VNF i doit être placée sur un serveur $\leq$ à celui de la VNF j
Paramètres	
$t_j > 0$	Charge de traitement de la VNF j (CPU μs par paquet)
m	Nombre de serveurs physiques disponibles
Variables de décision	
$x_{jk} \in \{0, 1\}$	$= 1$ si la VNF j est déployée sur le serveur k
$s_j \in \{1, \ldots, m\}$	Indice du serveur sur lequel la VNF j est déployée
$CT \geq 0$	Latence goulot (charge maximale des serveurs), à minimiser

2.2 Formulation MILP (Modèle M1)

M1

$$\min_{x,s,CT} \quad CT \tag{M1.1}$$

$$\text{s.c.} \quad \sum_{k\in\mathcal{K}} x_{jk} = 1 \quad \forall j \in \mathcal{J} \quad \text{[une seule affectation]} \tag{M1.2}$$

$$s_j = \sum_{k\in\mathcal{K}} k x_{jk} \quad \forall j \in \mathcal{J} \quad \text{[indice serveur]} \tag{M1.3}$$

$$s_i \leq s_j \quad \forall (i,j) \in P \quad \text{[précédence]} \tag{M1.4}$$

$$L_k = \sum_{j\in\mathcal{J}} t_j x_{jk} \quad \forall k \in \mathcal{K} \quad \text{[charge serveur]} \tag{M1.5}$$

$$L_k \leq CT \quad \forall k \in \mathcal{K} \quad \text{[goulot]} \tag{M1.6}$$

La contrainte (2) garantit que chaque VNF est affectée à exactement un serveur. **La contrainte (3)** définit l'indice de serveur s_j de chaque VNF comme étant l'indice k du serveur auquel elle est affectée ; comme un seul x_{jk} vaut 1, cette définition est bien posée.

La contrainte (4) est la contrainte d'ordre de la chaîne : si la VNF i doit être traitée avant la VNF j, alors le serveur de i doit avoir un indice inférieur ou égal à celui de j. Cette inégalité encode directement la dépendance du DAG et constitue la contrainte structurelle clé qui rend le problème non trivial.

Les contraintes (5)–(6) définissent la latence goulot minimisée en (1).

Remarque sur la contrainte d'ordre

La contrainte (4) impose que $s_i \leq s_j$, et non $s_i < s_j$. Deux VNF dépendantes peuvent être co-localisées sur le même serveur ($s_i = s_j$) : au sein d'un serveur, les paquets sont traités séquentiellement selon l'ordre de la chaîne par le pipeline local des VNF. La contrainte interdit uniquement de placer la VNF j sur un serveur antérieur à celui de sa VNF prédécesseure i.

2.3 Jeu de données de benchmark SALBPGen

Vous évaluerez votre solveur sur les instances de benchmark **SALBPGen** [3], qui constituent la référence standard pour cette classe de problèmes. Bien que générées initialement pour la production, elles sont structurellement identiques aux instances de chaînes VNF : chaque instance fournit n, m, les charges t_j et un DAG de précédence P.

Le paramètre *order-strength* (OS $\in [0,1]$) contrôle la densité du DAG : OS $= 0$ signifie aucune dépendance (placement trivial), OS $= 1$ correspond à une chaîne totalement linéaire (contrainte maximale). Les valeurs intermédiaires modélisent des SFC réalistes avec dépendances partielles.

Catégorie	$n = \lvert\mathcal{J}\rvert$	**Serveurs** m	**Instances**	**Plage de OS**
Petite	20	variable	20	0.2 – 0.9
Moyenne	50	variable	2	0.2 – 0.9
Grande	100	variable	2	0.2 – 0.9

Téléchargez les instances depuis ce lien ou utilisez le script de génération disponible sur la plateforme du cours.

2.4 Exigences d'implémentation

1. Écrire un parseur pour le format SALBPGen. Le réutiliser dans toutes les phases.
2. Implémenter le modèle M1 avec `gurobipy`. Utiliser les logs Gurobi pour capturer : temps de résolution, CT optimal (ou meilleure borne), gap d'optimalité (%), nombre de nœuds B&B.
3. Résoudre les 20 instances avec une limite de **30 minutes** par instance.
4. Produire un tableau de résultats et un graphique du temps de résolution en fonction de la taille et de l'order-strength.

2.5 Attendus du rapport pour la phase 1

- Décrire le modèle Gurobi : variables, contraintes et fonction objectif. Signaler les éventuels problèmes numériques.
- Discuter le comportement du solveur selon la taille des instances et la valeur de OS. Quelles propriétés structurelles rendent le problème plus difficile ?
- Identifier les instances résolues à optimalité prouvée dans le temps imparti.

Phase 2 — Heuristiques, métaheuristiques et renforcement exact

2.6 Motivation

Les solveurs MILP passent difficilement à l'échelle au-delà de quelques centaines de variables pour des instances difficiles. Deux stratégies complémentaires existent : (a) des *algorithmes approximatifs rapides* qui trouvent de bonnes solutions, et (b) le *renforcement de la formulation* qui améliore la relaxation LP et réduit l'arbre de Branch-and-Bound.

2.7 Partie A — Trois méthodes heuristiques ou métaheuristiques

Implémentez **trois méthodes distinctes**. Au moins une doit être une *métaheuristique* (avec aléatoire ou population). Pour chaque méthode, justifiez les choix et les hyperparamètres.

Pour chaque heuristique et chaque instance, rapporter : coût (CT), gap d'optimalité et temps CPU. Résumer dans un tableau comparatif.

$$\text{Gap} = \frac{CT_{\text{heuristique}} - CT^*}{CT^*} \times 100\%$$

2.8 Attendus du rapport pour la phase 2

- Décrire chaque heuristique et son réglage.
- Fournir le tableau comparatif complet.
- Identifier les instances où l'heuristique atteint l'optimum. Quelles propriétés les rendent faciles ?

Phase 3 — Placement de VNF avec prise en compte de l'énergie (EVNF-P)

2.9 Motivation : le défi énergétique dans les infrastructures NFV

La NFV transfère les charges de travail réseau vers des serveurs standards, mais cela s'accompagne d'un coût énergétique important. Les centres de données représentent déjà environ 1–2% de la consommation mondiale d'électricité [5], et cette part augmente avec le déploiement de la 5G et de l'edge computing. Un problème opérationnel critique est celui des points chauds thermiques : lorsqu'un serveur concentre des VNFs énergivores, la température de son CPU augmente fortement, ce qui dégrade les performances, réduit la durée de vie du matériel et déclenche des réponses de refroidissement coûteuses. Les opérateurs de centres de données estiment que **le refroidissement à lui seul représente 30–40% de la dépense énergétique totale** [6], et un mauvais équilibrage du placement des VNF est un facteur direct de cette surconsommation.

Cela crée une forte incitation à intégrer directement des critères énergétiques dans la décision de placement. Minimiser le pic de stress énergétique des serveurs — c'est-à-dire le serveur le plus chargé thermiquement — garantit qu'*aucun serveur ne devienne un goulot d'étranglement du point de vue de la dissipation thermique*, exactement comme on cherche à éviter qu'un seul serveur ne devienne le goulot d'étranglement en termes de latence de traitement.

2.10 L'intensité énergétique d'une VNF

Toutes les VNFs ne consomment pas la même puissance par unité de charge de traitement. Un pare-feu simple basé sur l'appariement de paquets sollicite peu le CPU ; un moteur d'inspection approfondie des paquets (DPI) effectuant des correspondances d'expressions régulières à débit de ligne sollicite fortement le CPU et génère une chaleur importante par paquet traité.

Nous caractérisons chaque VNF $j \in \mathcal{J}$ par un score d'intensité énergétique fixe $e_j \in \{1, \ldots, 15\}$, représentant sa consommation d'énergie normalisée par unité de charge CPU. Ce score est supposé être mesuré hors ligne (par profilage ou à partir des spécifications fournisseurs) et fourni en entrée.

Score e_j	**Niveau énergétique**	**Types typiques de VNF**	**Action requise**
1	Négligeable	ACL simple, routage statique	Aucune
2 – 4	Faible	NAT, répartiteur de charge L4 basique	Surveiller
5 – 8	Moyen	Pare-feu avec états, IDS	Planifier la capacité
9 – 12	Élevé	Terminaison TLS, DPI	Éviter la concentration
13 – 15	Très élevé	Détection d'anomalies basée sur le ML, transcodage vCDN	Répartir en urgence

2.11 Stress énergétique au niveau du serveur : agrégation pondérée par le temps

Une fois les VNFs placées sur les serveurs, nous avons besoin d'un nombre unique pour représenter le stress énergétique de chaque serveur. L'approche la plus simple consiste à sommer les scores énergétiques : $\sum_j e_j x_{jk}$. Cependant, cela est trompeur pour la même raison qu'une simple somme de scores est trompeuse dans tout contexte d'équilibrage de charge : elle ignore *combien de temps* chaque VNF fonctionne réellement.

Exemple. Le serveur k héberge deux VNFs : la VNF-A (charge $t_A = 2\,\mu$s, score $e_A = 12$, un module DPI qui fonctionne rarement) et la VNF-B (charge $t_B = 58\,\mu$s, score $e_B = 2$, un NAT simple qui

traite la majeure partie du trafic). La somme donne $12 + 2 = 14$, ce qui suggère un stress proche du maximum. Pourtant, la VNF-A ne représente que 3% du temps de traitement du serveur ; le stress réaliste est dominé par la VNF-B, de faible intensité énergétique.

Nous adoptons donc un score énergétique moyen pondéré par la charge comme métrique du stress au niveau du serveur. Pour un serveur $k \in \mathcal{K}$:

$$E_k = \frac{\sum_{j \in \mathcal{J}} e_j \, t_j \, x_{jk}}{\sum_{j \in \mathcal{J}} t_j \, x_{jk}} = \frac{N_k}{L_k} \tag{1}$$

où N_k est la charge de traitement pondérée par l'énergie et L_k est la charge totale de traitement définie en phase 1. Pour l'exemple ci-dessus : $E_k = (12 \times 2 + 2 \times 58)/60 \approx 2.33$, ce qui reflète fidèlement le faible stress énergétique global.

2.12 Objectif : minimiser le pic de stress énergétique des serveurs

Dans une SFC déployée, chaque serveur traite continuellement les paquets via les VNFs qui lui sont affectées. Faire la moyenne du stress énergétique entre serveurs peut laisser subsister un serveur comme point chaud thermique permanent. Nous adoptons donc le critère min-max :

$$\text{minimiser} \quad E_{\max} = \max_{k \in \mathcal{K}} E_k.$$

Minimiser $E_{\max}$ garantit qu'*aucun serveur unique* ne supporte une part disproportionnée de la charge énergétique. Cela permet un refroidissement plus uniforme, réduit le risque de bridage thermique et améliore la durée de vie globale de l'infrastructure.

2.13 Le modèle EVNF-P

Étant donné une borne de latence goulot $\bar{C}$ (obtenue à partir de la phase 1, avec par exemple $\bar{C} = 1.05 * CT$), le modèle EVNF-P (*Energy-aware VNF Placement*) minimise le pic de stress énergétique des serveurs sous les contraintes de placement, d'ordre et de latence. Les notations supplémentaires sont rassemblées dans le Tableau 1.

Paramètres supplémentaires	
$e_j \in \{1, \ldots, 15\}$	Score d'intensité énergétique de la VNF j
$E_{\min} = \min_j e_j$	Score minimal dans l'instance
$E_{\max\text{-bound}} = \max_j e_j$	Score maximal dans l'instance
$\bar{C} \geq CT^*$	Borne de latence (entrée managériale)
Variables de décision supplémentaires	
E_k	Score énergétique moyen pondéré par la charge du serveur k
$E_{\max}$	Pic de stress énergétique des serveurs (objectif)

Table 1: Notations supplémentaires pour la phase 3.

EVNF-P

$$\min_{x,s,E_k,E_{\max}} \quad E_{\max} \tag{E.1}$$

$$\text{s.t.} \quad \sum_{k\in\mathcal{K}} x_{jk} = 1 \qquad \forall\, j \in \mathcal{J} \tag{E.2}$$

$$s_j = \sum_{k\in\mathcal{K}} k \cdot x_{jk} \qquad \forall\, j \in \mathcal{J} \quad \text{[indice du serveur]} \tag{E.3}$$

$$s_i \le s_j \qquad \forall\, (i,j) \in P \quad \text{[ordre de la chaîne]} \tag{E.4}$$

$$L_k = \sum_{j\in\mathcal{J}} t_j\, x_{jk} \qquad \forall\, k \in \mathcal{K} \tag{E.5}$$

$$L_k \le \bar{C} \qquad \forall\, k \in \mathcal{K} \quad \text{[borne de latence]} \tag{E.6}$$

$$E_k \cdot L_k = \sum_{j\in\mathcal{J}} e_j\, t_j\, x_{jk} \qquad \forall\, k \in \mathcal{K} \quad \leftarrow \text{non linéaire} \tag{E.7}$$

$$E_k \le E_{\max} \qquad \forall\, k \in \mathcal{K} \tag{E.8}$$

$$x_{jk} \in \{0,1\}, \quad s_j \in \{1,\dots,m\} \qquad \forall\, j \in \mathcal{J},\, k \in \mathcal{K} \tag{E.9}$$

$$L_k \ge 0, \quad E_{\min} \le E_k \le E_{\max}^{\text{bound}} \qquad \forall\, k \in \mathcal{K} \tag{E.10}$$

Source de la non-linéarité

La contrainte (15) définit E_k via le rapport N_k/L_k (ou, de manière équivalente, le produit $E_k \cdot L_k = N_k$). En substituant $L_k = \sum_j t_j x_{jk}$, cela fait apparaître des *termes bilinéaires* $E_k \cdot x_{jk}$ — le produit d'une variable continue et d'une variable binaire. Cela fait de EVNF-P un programme non linéaire en nombres entiers mixtes que les solveurs MILP standards ne peuvent pas traiter directement.

Observation structurelle clé

Les termes bilinéaires $E_k \cdot x_{jk}$ impliquent une variable *continue* (E_k, bornée par $E_{\min} \le E_k \le E_{\max\text{-bound}}$) et une variable *binaire* ($x_{jk} \in \{0,1\}$).
Lorsqu'**l'un des facteurs est binaire**, la relaxation de McCormick du produit est *exacte* : elle n'introduit aucune erreur d'approximation, et les quatre inégalités de McCormick caractérisent exactement le produit sur l'ensemble faisable entier. C'est la propriété clé qui permet une reformulation MILP exacte.

2.14 Votre travail en phase 3

Implémentez **au moins une** approche de résolution pour EVNF-P. Deux directions sont proposées ; vous pouvez également proposer la vôtre.

Option A — MILP exact via linéarisation de McCormick

Introduisez des variables continues auxiliaires $Z_{jk} = E_k \cdot x_{jk}$ et remplacez chaque terme bilinéaire par son enveloppe de McCormick.

Option B — Approche heuristique

Adaptez une ou plusieurs de vos heuristiques ou métaheuristiques de la phase 2 afin de cibler $E_{\max}$ au lieu de CT. Évaluez la qualité des solutions par rapport aux résultats exacts lorsqu'ils sont disponibles. Discutez le compromis entre latence et stress énergétique.

Exigences pour toutes les soumissions

1. **Validation numérique sur l'instance illustrative** (Section 2.15) : reproduire les deux valeurs de $E_{\max}$.
2. **Analyse du compromis latence–énergie.** Que devient CT lorsque vous optimisez le stress énergétique ? Peut-on toujours améliorer $E_{\max}$ sans violer $L_k \leq \bar{C}$? Fournissez au moins une instance où il existe une véritable tension entre les deux objectifs.
3. **Analyse de sensibilité des instances.** Comment n, m, l'order-strength et la dispersion des scores énergétiques influencent-ils la valeur optimale de $E_{\max}$?

2.15 Exemple illustratif (point de validation)

Nous considérons une SFC comportant $n = 8$ modules VNF et $m = 3$ serveurs physiques. Les charges de traitement t_j et les scores d'intensité énergétique e_j sont donnés dans le Tableau 2. Le DAG de précédence est présenté dans la Figure **??**.

VNF j	Module	Fonction	t_j (μs)	e_j (score)
1	FW	Pare-feu avec états	8	7
2	NAT	Traduction d'adresses réseau	10	3
3	IPS	Système de prévention d'intrusion	1	12
4	DPI	Inspection approfondie des paquets	9	10
5	IDS	Système de détection d'intrusion	9	5
6	TLS	Terminaison TLS	3	14
7	LB	Répartiteur de charge	2	9
8	TC	Contrôleur de trafic	1	4

Table 2: Modules VNF, charges de traitement et scores d'intensité énergétique pour l'instance illustrative à 8 VNFs.

Les arcs de précédence sont :

$$P = \{(1,3),(1,6),(2,4),(2,5),(3,7),(4,7),(5,7),(6,8),(7,8)\}.$$

Cible de validation A — placement optimal de M1

Le modèle M1 (minimiser CT) donne le placement optimal suivant avec $\bar{C} = 17$:

$$S_1 = \{NAT\}, \quad S_2 = \{FW, DPI\}, \quad S_3 = \{IPS, IDS, TLS, LB, TC\}.$$

Charges des serveurs : $L_1 = 10$, $L_2 = 17$, $L_3 = 16$, donc $CT^* = 17$.

Le stress énergétique pondéré par la charge pour ce placement est :

$$
\begin{aligned}
E_1 &= \frac{10 \cdot 3}{10} = 3, \\
E_2 &= \frac{8 \cdot 7 + 9 \cdot 10}{17} = \frac{146}{17} \approx 8.59, \\
E_3 &= \frac{1 \cdot 12 + 9 \cdot 5 + 3 \cdot 14 + 2 \cdot 9 + 1 \cdot 4}{16} = \frac{121}{16} = 7.56.
\end{aligned}
$$

$$\boxed{E_{\max} = \max(3,\ 8.59,\ 7.56) \approx 8.59.}$$

Le serveur S_2 constitue le point chaud thermique : il héberge le module DPI, très énergivore, en plus du FW lourd, ce qui conduit à un score pondéré élevé.

Cible de validation B — placement optimal de EVNF-P

Le modèle EVNF-P donne le placement suivant :

$$S_1 = \{FW\}, \quad S_2 = \{NAT, IPS, DPI, IDS, TLS, LB\}, \quad S_3 = \{TC\}.$$

Charges des serveurs : $L_1 = 8$, $L_2 = 34$, $L_3 = 1$.

$$
\begin{aligned}
E_1 &= \frac{8 \cdot 7}{8} = 7, \\
E_2 &= \frac{10 \cdot 3 + 1 \cdot 12 + 9 \cdot 10 + 9 \cdot 5 + 3 \cdot 14 + 2 \cdot 9}{34} = \frac{237}{34} \approx 6.97, \\
E_3 &= \frac{1 \cdot 4}{1} = 4.
\end{aligned}
$$

$$\boxed{E_{\max} = \max(7,\ 6.97,\ 4) = 7.}$$

En concentrant de nombreuses VNFs d'intensités variées sur S_2, l'effet de moyenne réduit le pic de stress. La contrainte de goulot $L_k \leq 34$ est satisfaite pour $\bar{C} = 34$; la latence augmente, mais le point chaud thermique est éliminé.

Ce que cet exemple illustre

Il existe un véritable **compromis** entre latence et équilibre énergétique. Le placement optimal selon M1 ($CT = 17$) concentre des VNFs de forte intensité sur le serveur S_2 et produit $E_{\max} \approx 8.59$. Le placement optimal du point de vue énergétique répartit la charge plus uniformément ($E_{\max} = 7$), mais nécessite une borne de latence plus relâchée ($\bar{C} = 34$). En pratique, l'opérateur réseau fixe $\bar{C}$ en fonction des exigences de SLA, et EVNF-P trouve alors le placement énergétiquement optimal sous cette contrainte.

2.16 Attribution des scores énergétiques pour le benchmark

Les instances SALBPGen standards ne contiennent pas de scores énergétiques. Générez les scores comme suit : tirez chaque e_j uniformément dans $\{1, \ldots, 15\}$ en utilisant la **graine aléatoire 42**. Indiquez explicitement cette graine dans votre code et dans votre rapport.

2.17 Attendus du rapport pour la phase 3

- Présentez le modèle EVNF-P et expliquez chaque contrainte en langage clair. Utilisez le vocabulaire NFV de manière cohérente tout au long du document.
- Si vous implémentez l'option A : dérivez les quatre inégalités de McCormick et discutez les limites selon les instances.
- Reproduisez exactement les deux cibles de validation de la Section 2.15.
- Présentez les résultats de benchmark : valeurs de $E_{\max}$, temps de résolution, et comparaison avec les valeurs de CT^* de la phase 1 pour toutes les instances utilisées en phase 1.
- Discutez l'interprétation managériale : que garantit la minimisation de $E_{\max}$ pour l'opérateur d'infrastructure ? Quelles sont les limites de cette formulation mono-objectif ?

References

[1] ETSI ISG NFV (2012–2023). *Network Functions Virtualisation — Architectural Framework.* ETSI GS NFV 002. https://www.etsi.org/technologies/nfv

[2] Medhat A.M. et al. (2017). Service function chaining in next generation networks: State of the art and research challenges. *IEEE Communications Magazine*, 55(2), 216–223.

[3] Otto A., Otto C., Scholl A. (2013). Systematic data generation and test design for solution algorithms on the example of SALBPGen. *European Journal of Operational Research*, 228, 33–45. https://doi.org/10.1016/j.ejor.2012.12.029

[4] Boysen N., Schulze P., Scholl A. (2022). Assembly line balancing: What happened in the last fifteen years? *European Journal of Operational Research*, 301, 797–814.

[5] International Energy Agency (2022). *Data Centres and Data Transmission Networks.* https://www.iea.org/reports/data-centres-and-data-transmission-networks

[6] Shehabi A. et al. (2016). *United States Data Center Energy Usage Report.* Lawrence Berkeley National Laboratory, LBNL-1005775.